Java 入门笔记

第一课 · 基础概念、开发环境与程序结构

学习目标	掌握 Java 的运行原理，分清 JDK、JRE 与 JVM，能够编译并运行第一个 Java 程序。

目录

1. Java 是什么 2

2. Java 的主要特点 2

3. Java 的应用平台 3

4. JDK、JRE 与 JVM 3

5. 安装与配置 JDK 4

6. Java 程序的开发流程 5

7. 第一个 Java 程序 5

8. Java 应用程序的基本结构 6

9. 编程风格与开发工具 7

10. 标识符与命名规范 8

11. 注释 8

12. 初学阶段常见易错点 8

13. 本课小结 9

整理日期：2026 年 9 月 15 日

1. Java 是什么

Java 是一种高级、面向对象、跨平台的编程语言。它最初由 Sun Microsystems 推出，后来由 Oracle 维护。

Java 不仅指一门编程语言，也代表一整套开发与运行平台。常见应用包括：

- 后端服务与大型企业系统
- Android 应用（目前也常用 Kotlin）
- 大数据生态，如 Hadoop、Spark
- 桌面工具、开发工具和嵌入式设备

2. Java 的主要特点

2.1 简单易学

Java 语法与 C/C++ 相似，但移除了指针运算、多继承、手动释放内存等容易出错的机制。

“简单”是相对而言的：入门语法清晰，但完整的 Java 生态仍需要持续学习。

2.2 面向对象

Java 程序通常用对象描述现实世界中的事物：

```
现实问题（学生、课程、订单）
      ↓ 抽象
属性 + 行为
      ↓ 编程实现
类与对象
```

面向对象的三大特征：

- 封装：把数据和操作数据的方法放在一起，并隐藏不必要的实现细节。
- 继承：子类复用父类已有的属性和行为。
- 多态：同一个接口或父类引用，可以表现出不同的具体实现。

> 类是对象的模板，对象是类的具体实例。

2.3 平台无关

Java 源代码先被编译为与具体操作系统无关的字节码，再交给对应平台上的 JVM 执行。

```
源文件 .java
   ↓ javac 编译
字节码 .class
   ↓ JVM 解释执行或即时编译
机器指令
```

因此 Java 常被概括为：一次编写，到处运行（Write Once, Run Anywhere）。

需要注意：跨平台的是 Java 字节码，不是 JVM。Windows、macOS 和 Linux 需要安装各自平台对应的 JVM。

2.4 安全性

Java 不允许直接操作内存地址，并通过类加载、字节码校验、访问控制等机制提高程序安全性。

2.5 自动内存管理

对象通常由 JVM 分配内存；不再使用的对象由垃圾回收器（GC）自动回收。

GC 减少了手动管理内存的负担，但不能完全杜绝内存泄漏。例如，程序一直持有某个无用对象的引用时，该对象仍无法被回收。

2.6 多线程

Java 内置多线程支持，一个程序可以并发处理多项任务。例如：一边接收网络请求，一边读写文件。

2.7 动态性

JVM 可以在程序运行期间按需加载类。Java 还支持反射等动态机制。

“动态加载”不等于“内存开销一定小”；它主要表示类可以在运行时被加载、链接和使用。

2.8 健壮性

Java 提供强类型检查、异常处理、数组越界检查和自动垃圾回收等机制，帮助减少常见错误。

3. Java 的应用平台

- Java SE（Standard Edition）：Java 标准版，包含核心语法、集合、I/O、多线程等基础内容，初学者主要学习这一部分。
- Jakarta EE（原 Java EE）：面向企业级 Web 和分布式应用的规范与技术体系。
- Java ME（Micro Edition）：面向资源受限设备的 Java 平台，现在较少用于常规应用开发。

当前学习重点：Java SE。

4. JDK、JRE 与 JVM

4.1 JVM

JVM（Java Virtual Machine，Java 虚拟机）是字节码的执行引擎，负责加载和运行 .class 文件，并管理内存、垃圾回收等。

4.2 JRE

JRE（Java Runtime Environment，Java 运行时环境）包含 JVM 和运行 Java 程序所需的核心类库。

4.3 JDK

JDK（Java Development Kit，Java 开发工具包）包含编译、运行、调试 Java 程序所需的工具。

三者关系可以简化为：

```
JDK = 开发工具 + Java 运行环境
Java 运行环境 = JVM + 核心类库
```

现代 JDK 的发行方式与早期版本有所不同，通常直接安装 JDK 即可完成开发和运行。

4.4 常用命令

命令	作用
java -version	查看当前 Java 运行环境的版本
javac -version	查看 Java 编译器版本
javac 文件名.java	把源文件编译为字节码文件
java 类名	启动 JVM 并运行指定类，不写 .class 后缀
jar	创建、查看或解压 JAR 包
javadoc	根据注释生成 API 文档
jdb	Java 命令行调试器

5. 安装与配置 JDK

5.1 安装

可以从 Oracle JDK、OpenJDK 或其他可信的 JDK 发行版下载安装。学习时建议优先选择课程要求的版本或长期支持版（LTS）。

安装完成后，在终端执行：

```
java -version
javac -version
```

两个命令都能正常显示版本，说明安装基本成功。

5.2 JAVA_HOME

JAVA_HOME 应指向 JDK 的安装根目录，而不是 bin 目录。

macOS/Linux 示例：

```
export JAVA_HOME=$(/usr/libexec/java_home)
export PATH=$JAVA_HOME/bin:$PATH
```

Windows 示例：

```
JAVA_HOME = JDK 安装目录
Path 中添加：%JAVA_HOME%\bin
```

新版本 JDK 的日常开发通常不需要手动设置 CLASSPATH。Java 默认会在当前目录等位置查找类；只有项目或旧工具明确要求时才配置它。

6. Java 程序的开发流程

```
1. 编写源文件（.java）
2. 使用 javac 编译
3. 生成字节码文件（.class）
4. 使用 java 命令运行
```

示例命令：

```
javac HelloWorld.java
java HelloWorld
```

7. 第一个 Java 程序

文件名：HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

7.1 代码说明

- public class HelloWorld：定义一个公开类，类名为 HelloWorld。
- public static void main(String[] args)：Java 程序的入口方法。
- System.out.println(...)：向控制台输出内容，并在末尾换行。
- 每条普通语句通常以英文分号；结尾。
- Java 严格区分大小写，例如 String 不能写成 string。

7.2 public 类与文件名

一个源文件中最多只能有一个 public 顶级类。若存在 public 类，文件名必须与类名完全一致：

```
public class HelloWorld → HelloWorld.java
```

7.3 print 与 println

```
System.out.print("Hello ");
System.out.println("Java");
```

输出结果：

```
Hello Java
```

- print：输出后不换行。
- println：输出后换行。

8. Java 应用程序的基本结构

8.1 源文件、类与主类

一个 Java 应用程序由一个或多个类组成。这些类既可以写在同一个源文件中，也可以分布在多个源文件中。

```
Java 应用程序
├── HelloWorld.java（源文件）
│   ├── public class HelloWorld（主类）
│   │   └── main 方法（程序入口）
│   └── class Helper（普通类）
└── Student.java（源文件）
    └── class Student（普通类）
```

基本规则：

- 一个 .java 源文件中可以定义多个顶级类。
- 一个源文件中最多只能有一个 public 顶级类。
- 如果源文件包含 public 顶级类，文件名必须与该类名完全一致。
- Java 应用程序启动时，需要指定一个包含 main 方法的类。
- 初学阶段通常把主类声明为 public，并将 main 方法写成标准形式。

```
public class Main {
    public static void main(String[] args) {
        System.out.println("程序从这里开始执行");
    }
}

class Helper {
    // 同一源文件中的普通类
}
```

> public 类、主类和 main 方法是三个相关但不同的概念：public 是访问权限，主类是启动时指定的类，main 方法是程序入口。

8.2 类、属性、方法与对象

```
class Student {
    String name; // 属性：描述对象的数据

    void study() { // 方法：描述对象的行为
        System.out.println(name + " is studying");
    }
}
```

创建并使用对象：

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.name = "Tom";
        student.study();
    }
}
```

其中：

- Student 是类。
- student 是引用变量。
- new Student() 创建了一个对象。
- student.name 和 student.study() 分别访问对象的属性与方法。

9. 编程风格与开发工具

9.1 大括号风格

常见的大括号书写风格有以下两种。它们只影响代码排版，不改变程序功能。

Allman 风格：左大括号单独占一行。

```
public class Example
{
    public static void main(String[] args)
    {
        System.out.println("Allman style");
    }
}
```

K&R（Kernighan and Ritchie）风格：左大括号放在语句末尾。

```
public class Example {
    public static void main(String[] args) {
        System.out.println("K&R style");
    }
}
```

Java 项目通常采用 K&R 风格。实际开发中应遵循团队统一的代码规范，并使用 IDE 自动格式化代码。

9.2 常用开发工具

- IntelliJ IDEA：功能完整、智能提示丰富，是常用的 Java IDE。
- Eclipse：经典的开源 Java IDE，插件生态成熟。
- Visual Studio Code：安装 Java 扩展后可以进行轻量级 Java 开发。

IDE 能帮助编写、运行和调试代码，但底层编译与运行仍依赖 JDK。

10. 标识符与命名规范

标识符用于给类、变量、方法等命名。

基本规则：

- 可以由字母、数字、下划线和美元符号组成。
- 不能以数字开头。
- 不能使用 Java 关键字，如 class、public、int。
- 严格区分大小写。

常见命名规范：

- 类名使用大驼峰：HelloWorld、StudentInfo。
- 变量名和方法名使用小驼峰：studentName、printInfo。
- 常量通常全部大写，单词间用下划线连接：MAX_VALUE。
- 包名通常全部小写：com.example.demo。

11. 注释

```
// 单行注释

/*
 多行注释
*/

/**
 * 文档注释，可配合 javadoc 使用
 */
```

注释用于解释代码，不会作为普通程序逻辑执行。

12. 初学阶段常见易错点

1. Java 没有供程序员直接运算的指针，但有引用；引用不等于 C/C++ 中的指针。
2. Java 严格区分大小写。
3. 如果源文件中有 public 顶级类，类名必须与源文件名完全一致。
4. main 方法的标准签名是 public static void main(String[] args)。

5 字符串使用英文双引号，单个字符使用英文单引号。
6 源代码中的括号和分号要使用英文符号。
7 javac 后写文件名和 .java 后缀；java 后写类名，不写 .class 后缀。
8 Java 字节码可以跨平台，JVM 本身需要区分平台。
9 GC 自动回收内存，但不能完全避免内存泄漏。
10 JIT（Just-In-Time）是 JVM 的即时编译机制，可把热点字节码编译成本地机器码，提高运行效率。
11 面向对象三大特征是封装、继承和多态。

13. 本课小结

- Java 源代码先编译为字节码，再由 JVM 执行。
- JDK 用于开发，JVM 负责执行字节码。
- Java 的核心优势包括面向对象、跨平台、安全、健壮和自动内存管理。
- 一个 Java 应用程序由一个或多个类组成，并通过 main 方法进入程序。
- public 顶级类、源文件名与程序入口之间的规则要分清。
- 编写代码时应保持统一的命名与大括号风格。
- 学习 Java 的第一步是理解类、对象、main 方法以及编译运行流程。